Warehouse Delivery Robot

Path finding with search algorithms: BFS, DFS, Greedy Best-First

Intelligent Systems - Task 1

Faculty of Computers and Data Science, Alexandria University

Team: Yahia Tarek

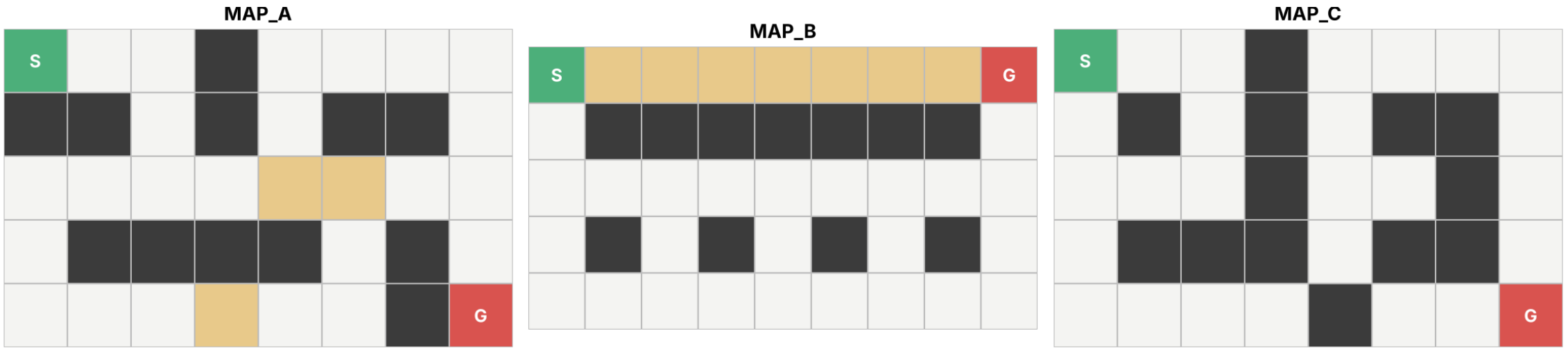
github.com/yahiat168/game-puzzle

The problem

Get the robot from S to G as cheaply as possible

- Grid map; the robot starts at S and must reach the goal G
- Moves one cell at a time: Up, Down, Left, Right
- Normal floor . costs 1 | Rough floor ~ costs 3 | Wall # blocked
- Path cost = sum of the cells entered (S not counted, G costs 1)

Green = start S, red = goal G, tan = rough floor, dark = wall



Three test maps

Each one tests something different

- MAP_A - a maze: the robot must wind around walls and cannot avoid all rough floor
- MAP_B - the trap: the shortest route is not the cheapest
  - Top row: 8 moves but 7 rough cells = cost 22
  - Detour through row 2: 12 moves, all normal floor = cost 12
- MAP_C - no solution: walls cut G off from S
  - A correct algorithm must stop and print "No path found"

Problem formulation

Modelled as a state-space search

Component	Definition
State	The robot's cell (row, col)
Initial state	Cell S, found by scanning the grid
Actions	Up, Down, Left, Right (one cell)
Transition	Allowed if inside the grid and not a wall #
Step cost	. = 1 ~ = 3 G = 1
Goal test	Current cell is G
Path cost	Sum of step costs from S to G

Algorithms implemented

From scratch in Python - no path-finding libraries

Algorithm	Type	Frontier	Picks next	Uses cost?
BFS	Uninformed	FIFO queue	Shallowest node	No
DFS	Uninformed	LIFO stack	Deepest node	No
Greedy Best-First	Informed	Priority queue h(n)	Closest-looking to G	No

- BFS: explores level by level, always finds the path with the fewest moves
- DFS: goes as deep as possible, backtracks at dead ends; result depends on move order
- Greedy: always expands the node with the smallest heuristic h(n)

Heuristic: Manhattan distance

Used by Greedy Best-First

$$h(n) = |row - row_G| + |col - col_G|$$

- Counts the minimum moves to G if there were no walls
- Fits the robot exactly: it can only move Up / Down / Left / Right
- Admissible: every move costs at least 1, so h(n) never overestimates
- Limitation: ignores walls and rough floor, and Greedy ignores the cost already paid

Results

Average of 200 runs per algorithm

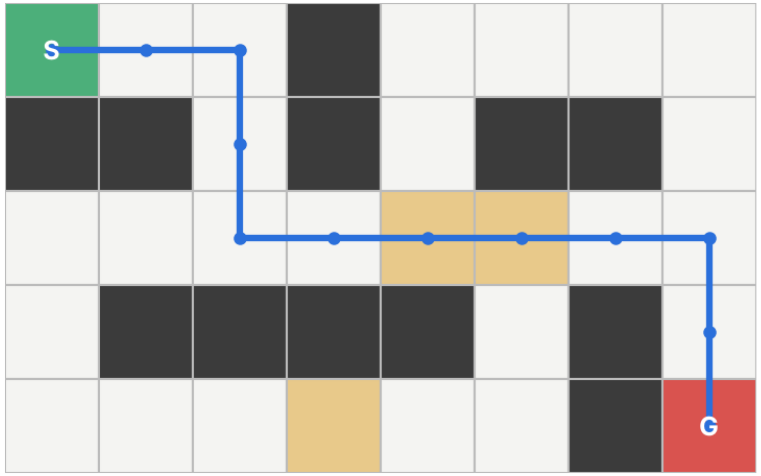
Map	Algorithm	Length	Cost	Expanded	Max frontier	Time (ms)
MAP_A	BFS	11	15	28	5	0.037
MAP_A	DFS	25	31	26	5	0.043
MAP_A	Greedy Best-First	11	15	14	5	0.033
MAP_B	BFS	8	22	25	4	0.041
MAP_B	DFS	20	20	24	7	0.062
MAP_B	Greedy Best-First	8	22	9	2	0.018
MAP_C	BFS	-	-	13	3	0.017
MAP_C	DFS	-	-	13	3	0.020
MAP_C	Greedy Best-First	-	-	13	3	0.021

'-' = No path found. Highlighted = fell into the MAP_B trap (cost 22 when a cost-12 path exists).

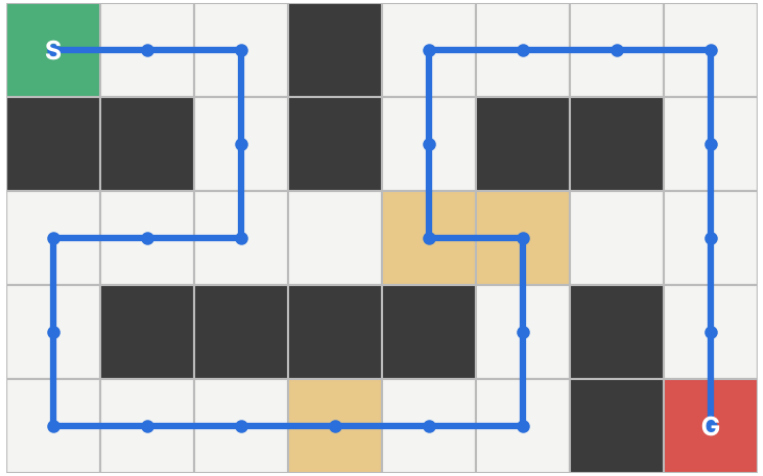
MAP_A paths

BFS and Greedy find the same path (cost 15); DFS takes a long detour (cost 31)

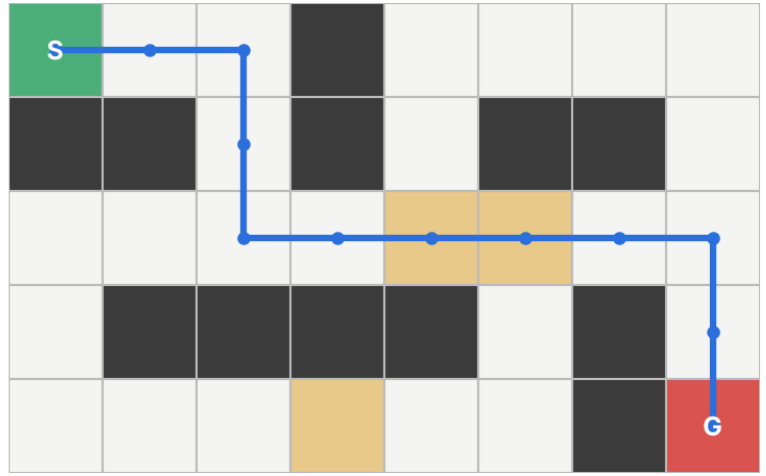
BFS | cost 15, 11 moves, 28 expanded



DFS | cost 31, 25 moves, 26 expanded



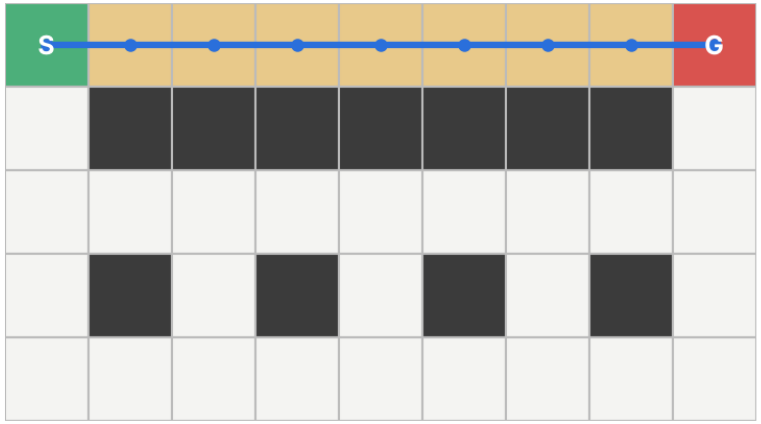
Greedy Best-First | cost 15, 11 moves, 14 expanded



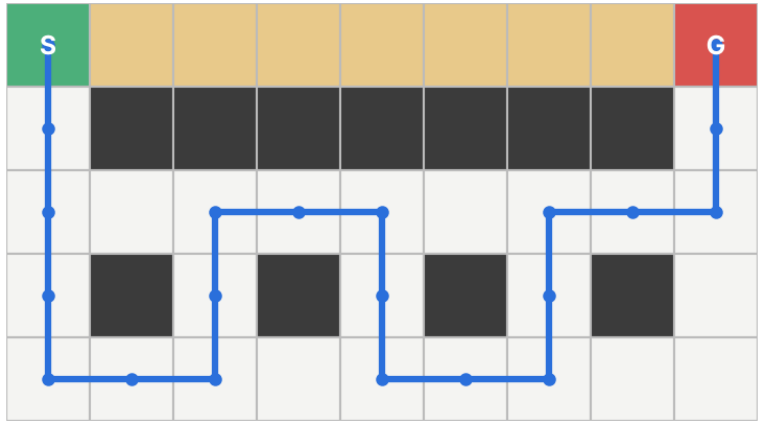
MAP_B: the trap

BFS and Greedy take the rough top row (cost 22); DFS zig-zags below (cost 20)

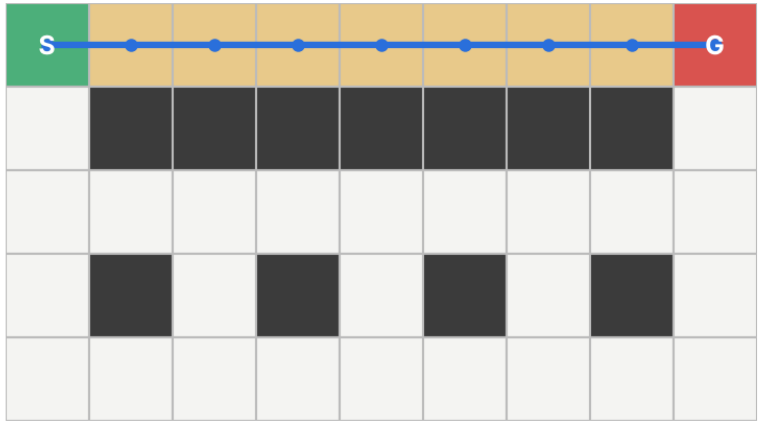
BFS | cost 22, 8 moves, 25 expanded



DFS | cost 20, 20 moves, 24 expanded



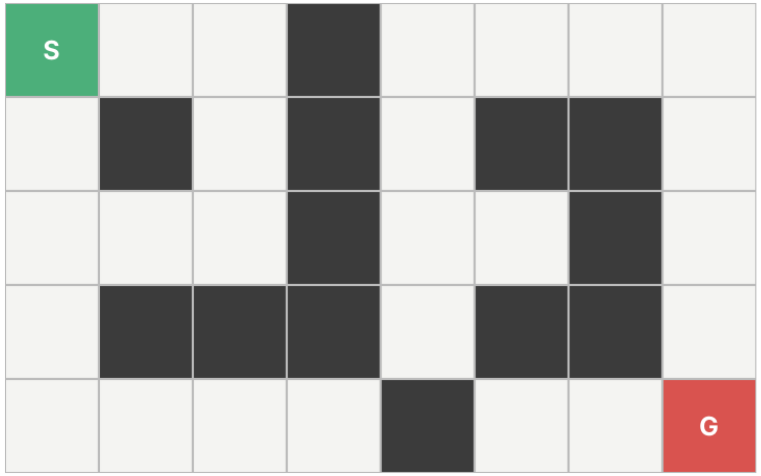
Greedy Best-First | cost 22, 8 moves, 9 expanded



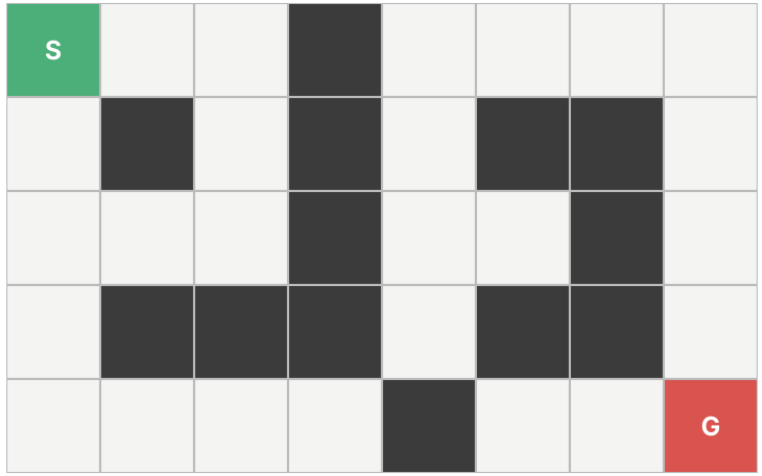
MAP_C: no solution

All three explore the same 13 reachable cells, then print "No path found"

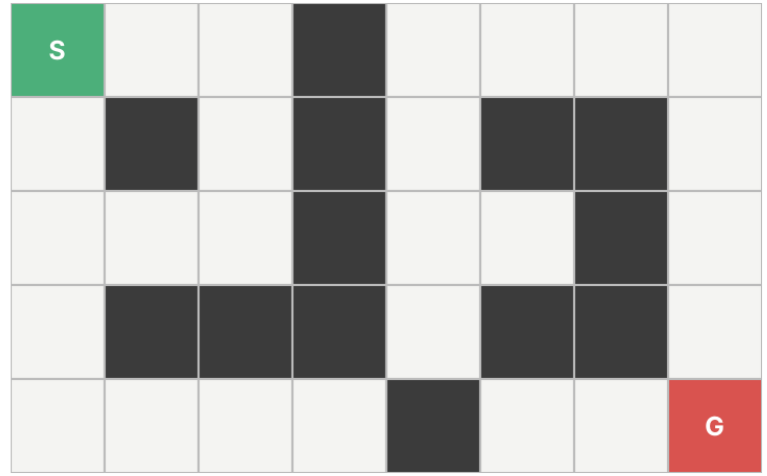
BFS | No path found (13 expanded)



DFS | No path found (13 expanded)



Greedy Best-First | No path found (13 expanded)



Comparison

Which algorithm suits the robot?

Question	BFS	DFS	Greedy	Winner
Cost on MAP_A	15	31	15	BFS = Greedy
Cost on MAP_B	22	20	22	DFS (by luck)
Nodes expanded A / B	28 / 25	26 / 24	14 / 9	Greedy
Max frontier A / B	5 / 4	5 / 7	5 / 2	Greedy

- Greedy expanded 50% fewer nodes than BFS on MAP_A and 64% fewer on MAP_B
- DFS beat the others on MAP_B only because it tries Down before Right
- None of the three uses cell costs, so none found the cost-12 path on MAP_B

Conclusion

Our choice: Greedy Best-First

- Greedy matched BFS on path cost, with the fewest expansions and the least memory
- DFS is unreliable: its path depends on move order (cost 31 vs 15 on MAP_A)
- MAP_B shows the shortest path is not always the cheapest
- All three correctly report "No path found" on MAP_C
- To guarantee the cheapest path, a cost-aware algorithm (UCS or A*) is needed

Run it: `python search_robot.py`

Change map: `GRID = MAP_B`

github.com/yahiat168/game-puzzle